DESIGN AND ANALYSIS OF ALGORITHM (CS-327)

LECTURE – 3

Example of Asymptotic Notations (Fruits List)

List of N items

Apple
Banana
Grapes
Orange
-
-
-
Strawberry

Alphabetically Sorted List

List of N items

Strawberry
Orange
Grapes
Apple
-
-
-
Banana

Unsorted List Random List

Case 1 :-

Suppose we want to search **"Banana"** in alphabetically sorted list so we can quickly search it because the list is an alphabetically sorted list. So it will take constant number of operations and a lower bound. So the complexity of searching **"Banana"** in alphabetically sorted list is **W(1) or O(1)**.

Case 2 :-

Suppose we want to search **"Banana"** in alphabetically sorted list and we have hundreds or thousands of elements starts with **"B"** so we will have to further break that list and it will take logarithmic time so the complexity of searching **"Banana"** is **O(logn)**.

Case 3 :-

Suppose we want to search **"Banana"** in unsorted list or random list so we will have to go through whole list because the list is not sorted so the complexity of searching banana is **O(n).**

Analysis :-

Alphabetically sorted list is randomly faster than an unsorted list or random list. So as far as time complexity is concerned.

W(1) has less growth than		O(logn)has less growth than		O(n)
W(1)	**<**	**O(logn)**	<	**O(n)**
Lower bound	<	Tight bound	<	Upperbound

Growth = 1/ Efficiency.

Example of Asymptotic Notations (Programming examples)

We will discuss the following topics.

1. Constant
2.Loop
3.Nested Loop
4.Sequential
5.Conditional
6.Log

1.Constant :-

```
void function(int n)
{
int i=0;
for (i=0 ; i<10 ; i++)        /* Whether i<10 or i <100,  or  i < 1000, we consider it constant
{                             and time complexity is O(1) */

Cout << i ;
}
}
```

2.Loop :-

(i)

```
void function(int n)
{
int i=0;
for(i=0 ; i<n ; i++)        // Loop will run n times so time complexity is O(n).
{
Cout << i ;
}
}
```

(ii)

```
void function(int n)
{
int i=0;
for(i=0 ; i<n ; i++)      /*  Loop will run n times, but because of break statement loop will execute
Cout << i ;                  only once so time complexity is O(1) */
 break ;
}
```

3. **Nested Loop :-**

(i)

```
void function(int n)
{
int i=0;
int j=0;
for(i=0 ; i < n ; i++)   //  Loop will run n times, so time  complexity is O(n)
for(j=0 ; j < n ; j++)   //  Loop will run n times,  so time complexity is O(n)
{
Cout << i ;
}
}
```

Note :- So according to rule when we have nested loop in same body so we multiply the complexity of both loops so the overall time complexity of above loops is **$O(n^2)$**.

(ii)

```
void function(int n)
{
int i=0;
int j=0;
for(i=0 ; i < n ; i++)   //  Loop will run n times, so time  complexity is O(n)
for(j=0 ; j < n ; j++)   //  Loop will run n times,  so time complexity is O(1)
break ;
}
```

Note :- So according to rule when we have nested loop in same body so we multiply the complexity of both loops so the overall time complexity of above loops is **O(n)**.

4. **Sequential :-**

```
void function(int n)
{
int i=0;
int j=0;
int k=0;

{
for(i=0  ;  i  <  n ;  i++)   //  Loop will run n times, so time  complexity is O(n)
for(j=0  ;  j  <  n  ; j++)   //  Loop will run n times,  so time complexity is O(n)
for (k=0;  K < n  ; K++) //  Loop will run n times, so time  complexity is  O(n)
}
{
for(i=0  ;  i  <  n ;  i++)   //  Loop will run n times, so time  complexity is O(n)
for(j=0  ;  j  <  n  ; j++)   //  Loop will run n times,  so time complexity is O(n)
}
{
for(i=0  ;  i  <  n ;  i++)   //  Loop will run n times, so time  complexity is O(n)

}
}
```

O(n^3)

O(n^2)

Note :- So when we have loops in different body so we add the complexity so by adding we get $n^3 + n^2 + n$, so by considering highest power(worst case) the overall complexity is **O(n^3)**.

5. **Conditional** :-

```
void function(int n)
{
int i=0;
int j=0;
int k=0;
if (n > 20)
{
for(i=0  ;  i  <  n ;  i++)   //  Loop will run n times, so time  complexity is O(n)
for(j=0  ;  j  <  n  ; j++)   //  Loop will run n times,  so time complexity is O(n)
for (k=0;  K < n  ; K++) //  Loop will run n times, so time  complexity is  O(n)
}
else if ((n >=10) && (n<=15))
{
for(i=0  ;  i  <  n ;  i++)   //  Loop will run n times, so time  complexity is O(n)
for(j=0  ;  j  <  n  ; j++)   //  Loop will run n times,  so time complexity is O(n)
}
else if (n<10)
{
for(i=0  ;  i  <  n ;  i++)   //  Loop will run n times, so time  complexity is O(n)

}
}
```

O(n^3)

O(n^2)

Note :-

So in above example of code the time complexity would be

Best case TC is **O(n)** < Average case TC is **O(n^2)** < Worst case TC is **O(n^3)**

Lower bound < Tight bound < upper bound

6. **Log** :-

```
void function(int n)
{
int i=1;
While (i<n)      // it will run till kth iteration so time complexity is O(log2 n)
{
i=i*2;
}
}
```